

Set 18: String Methods

Skill 18.1: Differentiate between a function and a string method

Skill 18.2: Explore formatting methods

Skill 18.3: Apply *split* to convert a string to a list

Skill 18.4: Apply *join* to combine strings

Skill 18.5: Apply *strip* to clean strings

Skill 18.6: Apply *replace* to modify strings

Skill 18.7: Apply *find* to search strings

Skill 18.8: Apply *format* to compose strings

Skill 18.1: Differentiate between a function and a string method

Skill 18.1 Concepts

Python comes with built-in *string methods* that give you the power to perform complicated tasks on strings very quickly and efficiently. These string methods allow you to change the case of a string, split a string into many smaller strings, join many small strings together into a larger string, and allow you to neatly combine changing variables with string outputs.

In the previous lesson, you worked with *len()*, which was a *function* that determined the number of characters in a string. This, while similar, was NOT a string *method*. String methods all have the same syntax,

```
string.name.string_method(arguments)
```

Unlike *len()*, which is called with a string as its argument, a string method is called at the end of a string and each one has its own method specific arguments.

Skill 18.1 Exercise 1

Skill 18.2: Explore formatting methods

Skill 18.2 Concepts

There are three string methods that can change the casing of a string. These are *lower()*, *upper()*, and *title()*.

- *lower()* returns the string with all lowercase characters.
- *upper()* returns the string with all uppercase characters.
- *title()* returns the string in title case, which means the first letter of each word is capitalized.

Below is an example of each,

Code	Output
<pre>fav_song = 'SmOoTH CrimNaL' fav_song_lower = fav_song.lower() fav_song_upper = fav_song.upper() fav_song_title = fav_song.title() print(fav_song_lower) print(fav_song_upper) print(fav_song_title) print(fav_song) #fav_song is unchanged</pre>	<pre>smooth crimnal SMOOTH CRIMNAL Smooth Crimnal SmOoTH CrimNaL</pre>

It's important to remember that string methods can only **create** new strings, they do not change the original string. This is illustrated in the example above.

[Skill 18.2 Exercise 1](#)

Skill 18.3: Apply *split* to convert a string to a list

Skill 18.3 Concepts

upper(), *lower()*, and *title()* are all performed on an existing string and produce a string in return. Let's take a look at a string method that returns a different datatype.

When *split()* is performed on a string, it returns a list of strings by splitting up the original string by a separator (also called a delimiter).

If you do not provide an argument for *split()* it will default to splitting at spaces. Below are some examples,

Code	Output
<pre>line_one = "The sky has given over" print(line_one.split()) print(line_one.split("e"))</pre>	<pre>['The', 'sky', 'has', 'given', 'over'] ['Th', ' sky has giv', 'n ov', 'r']</pre>

We can also split strings using *escape sequences*. Escape sequences are used to indicate that we want to split by something in a string that is not necessarily a character. The two escape sequences we will cover here are,

- `\n` Newline
- `\t` Horizontal Tab

Newline or `\n` will allow us to split a multi-line string by line breaks and `\t` will allow us to split a string by tabs. `\t` is particularly useful when dealing with certain datasets because it is not uncommon for data points to be separated by tabs.

Let's take a look at an example of splitting by an escape sequence,

Code
<pre>smooth_chorus = """And if you said, "This life ain't good enough." I would give my world to lift you up I could change my life to better suit your mood Because you're so smooth""" chorus_lines = smooth_chorus.split('\n') print(chorus_lines)</pre>
Output
<pre>['And if you said, "This life ain\'t good enough.', 'I would give my world to lift you up', 'I could change my life to better suit your mood', "Because you're so smooth"]</pre>

The new list contains each line of the original string as its own smaller string. Also, notice that Python automatically escaped the ' character in the first line and adjusted to double quotation marks to allow the apostrophe on last line when it created the new list.

[Skill 18.3 Exercises 1 and 2](#)

Skill 18.4: Apply join to combine strings

Skill 18.4 Concepts

Now that you've learned to break strings apart using *split()*, let's learn to put them back together using *join()*. *join()* is essentially the opposite of *split()*, it *joins* a list of strings together with a given delimiter. Below is an example,

Code
<pre>my_munequita = ['My', 'Spanish', 'Harlem', 'Mona', 'Lisa'] print(' '.join(my_munequita))</pre>
Output
<pre>My Spanish Harlem Mona Lisa</pre>

In the example above, we took the list of strings, *my_munequita*, and we joined it together with our delimiter, ' ', which is a space. The space is important if you are trying to build a sentence from words. In the example below the space between the quotes has been removed.

Code
<pre>my_munequita = ['My', 'Spanish', 'Harlem', 'Mona', 'Lisa'] print(''.join(my_munequita))</pre>
Output
MySpanishHarlemMonaLisa

join can be applied to combine lists with any string delimiter. One often used string is a comma , because then we can create a string of *comma-separated values*, or CSV

You'll often find data stored in CSVs because it is an efficient, simple file type used by popular programs like Excel or Google Spreadsheets.

Code
<pre>my_munequita = ['My', 'Spanish', 'Harlem', 'Mona', 'Lisa'] print(', '.join(my_munequita))</pre>
Output
My,Spanish,Harlem,Mona,Lisa

You can also join using *escape sequences* as the delimiter. Consider the following example,

Code
<pre>smooth_fifth_verse_lines = ['Well I\'m from the barrio', 'You hear my rhythm on your radio', 'You feel the turning of the world so soft and slow', 'Turning you \'round and \'round'] smooth_fifth_verse = '\n'.join(smooth_fifth_verse_lines) print(smooth_fifth_verse)</pre>
Output
<pre>Well I'm from the barrio You hear my rhythm on your radio You feel the turning of the world so soft and slow Turning you 'round and 'round</pre>

Skill 18.4 Exercise 1

Skill 18.5: Apply *strip* to clean strings

Skill 18.5 Concepts

When working with strings that come from real data, you will often find that the strings aren't super clean. You'll find lots of extra whitespace, unnecessary line breaks, and rogue tabs. Python provides a great method for cleaning strings: *strip()*.

Stripping a string removes all whitespace characters from the beginning and end. Consider the following example,

Code
<pre>featuring = " Rob Thomas " print(feating.strip())</pre>
Output
Rob Thomas

In the example above, all the whitespace on either side of the string has been stripped, but the whitespace in the middle has been preserved.

You can also use *strip()* with a character argument, which will strip that character from either end of the string. In the example below, we strip the *!* character but the white space remains.

Code
<pre>featuring = "!!!Rob Thomas !!!!!" print("",featuring.strip('!'),"")</pre>
Output
' Rob Thomas '

Skill 18.5 Exercises 1 & 2

Skill 18.6: Apply *replace* to modify strings

Skill 18.6 Concepts

The next string method we will cover is *replace()*. *replace()* takes two arguments and replaces all instances of the first argument in a string with the second argument.

Code
<pre>with_spaces = "You got the kind of loving that can be so smooth" with_underscores = with_spaces.replace(' ', '_') print(with_underscores)</pre>
Output
you got the kind of loving that can be so smooth

Above we used *replace()* to change every instance of a space in the string to be an underscore instead.

Skill 18.6 Exercise 1

Skill 18.7: Apply *find* to search strings

Skill 18.7 Concepts

Another interesting string method is *find()*.

find() takes a string as an argument and searches the string it was run on for that string. It then returns the first *index value* where that string is located.

Here's an example,

Code	Output
<pre>print('smooth'.find('t'))</pre>	4

We searched the string 'smooth' for the string 't' and found that it was at the fourth index spot, so *find()* returned 4.

You can also search for larger strings, and *find()* will return the index value of the first character of that string. If the string is not found, -1 is returned instead.

Code	Output
<pre>print("smooth".find('oo'))</pre>	2
<pre>print("smooth".find('x'))</pre>	-1

Finally, you specify where in the string you want to start searching from. In the example below, we are locating the comma immediately after the *load_time*.

Code	Output
<pre>log_line = "[INFO] user_id=42, load_time=351ms, status=OK" load_time = log_line.find("load_time=") time_end = log_line.find(",", load_time) print(time_end)</pre>	34

[Skill 18.7 Exercises 1 and 2](#)

Skill 18.8: Apply *format* to compose strings

Skill 18.8 Concepts

Python also provides a handy string method for including variables in strings. This method is *format()*. *format()* takes variables as an argument and includes them in the string that it is run on. You include {} marks as placeholders for where those variables will be imported.

Consider the following function,

```
def favorite_song_statement(song, artist):  
    return "My favorite song is {} by {}".format(song, artist)
```

The function *favorite_song_statement* takes two arguments, *song* and *artist*, then returns a string that includes both of the arguments and prints a sentence. Note: *format()* can take as many arguments as there are {} in the string it is run on, which in this case is two.

Here's an example of the function being run,

Code
<pre>def favorite_song_statement(song, artist): return "My favorite song is {} by {}".format(song, artist) print(favorite_song_statement("Smooth", "Santana"))</pre>
Output
My favorite song is Smooth by Santana

Now you may be asking yourself, I could have written this function using string concatenation instead of *.format()*, why is this method better? The answer is legibility and reusability. It is much easier to picture the end result *.format()* than it is to picture the end result of string concatenation and legibility is everything. You can also reuse the same base string with different variables, allowing you to cut down on unnecessary, hard to interpret code.

format() can be made even more legible for other people reading your code by including *keywords*. Previously, with *.format()*, you had to make sure that your variables appeared as arguments in the same order that you wanted them to appear in the string, which adds unnecessary complications when writing code.

By including keywords in the string, and in the arguments, you can remove that ambiguity. Let's look at an example.

Code
<pre>def favorite_song_statement(song, artist): return "My favorite song is {song} by {artist}".format(artist=artist, song=song) print(favorite_song_statement("Smooth", "Santana"))</pre>
Output
My favorite song is Smooth by Santana

In the above example, it is clear to anyone reading the string what it is supposed to return, they don't even need to look at the arguments of *format()* in order to get a clear understanding of what is supposed to happen. You can even reverse the order of artist and song in the code above and it will work the same way (we did this in the example above).

Skill 18.8 Exercise 1